

Automatização de Testes de Qualidade com Cypress

Apresentação de Estágio no Âmbito do Curso de Engenharia Informática

Ricardo Duarte Rodrigues
2022147797

julho 2025

INTRODUÇÃO
BRNDTS

- Fundada em 2024.
- Incubada no IPN, Coimbra.
- **Especialização:** Soluções para monetização de vídeos online, com foco em publicidade.
- **Visão:** Transformação da experiência publicitária com tecnologias avançadas.



INTRODUÇÃO
BRNDTS



INTRODUÇÃO
Objetivos do estágio

- Integrar um processo eficaz de QA na equipa de desenvolvimento;
- Implementar testes E2E com Cypress + Cucumber + TypeScript;
- Utilizar Zephyr + Jira para rastreabilidade;
- Automatizar execução de testes via Jenkins (CI/CD).

INTRODUÇÃO
Análise do problema

Pontos a resolver:

- Falta de estratégia estruturada de Quality Assurance (QA);
- Testes end-to-end (E2E) eram executados manualmente, sem padronização;
- Ausência de critérios de aceitação consistentes;
- Rastreabilidade de bugs e testes deficiente;
- Validações dependiam do conhecimento informal dos developers;
- Falta de métricas objetivas para medir a qualidade das entregas.

INTRODUÇÃO
Análise do problema (cont.)

Soluções:

- Definir e operacionalizar uma estratégia transversal de QA;
- Automatizar testes E2E com Cypress + Cucumber + TypeScript;
- Padronizar critérios de aceitação (Gherkin/BDD);
- Garantir rastreabilidade com Zephyr + Jira;
- Integrar os testes nas pipelines CI/CD (Jenkins);
- Criar documentação e relatórios de validação;
- Promover uma cultura de qualidade contínua na equipa.

INTRODUÇÃO

Ferramentas e Tecnologias

Desenvolvimento









Gestão de Projetos e Qualidade











INTRODUÇÃO

Porquê de usar essas tecnologias

- **Cypress:** Permite a execução rápida e fiável de testes end-to-end diretamente no browser, com excelente visibilidade e facilidade de debugging, para além de ser muito intuitiva e rápida aprendizagem.
- **Cucumber:** Permite escrever testes em linguagem natural, promovendo a comunicação entre membros técnicos e não técnicos da equipa e alinhando os testes com os critérios de aceitação definidos. Fácil integração com Cypress.
- **Jenkins:** Permite integrar os testes automatizados nas pipelines CI/CD, garantindo validações consistentes e automáticas.
- **Zephyr:** Essencial para garantir a rastreabilidade dos testes, permitindo organizar casos por funcionalidades, acompanhar execuções e gerar relatórios com métricas objetivas.
- **Jira, Confluence, Bitbucket, Git, Microsoft Teams:** Já eram utilizadas pela equipa.

INTRODUÇÃO
Organização Diária e Colaboração

Ambiente Ágil (Scrum):

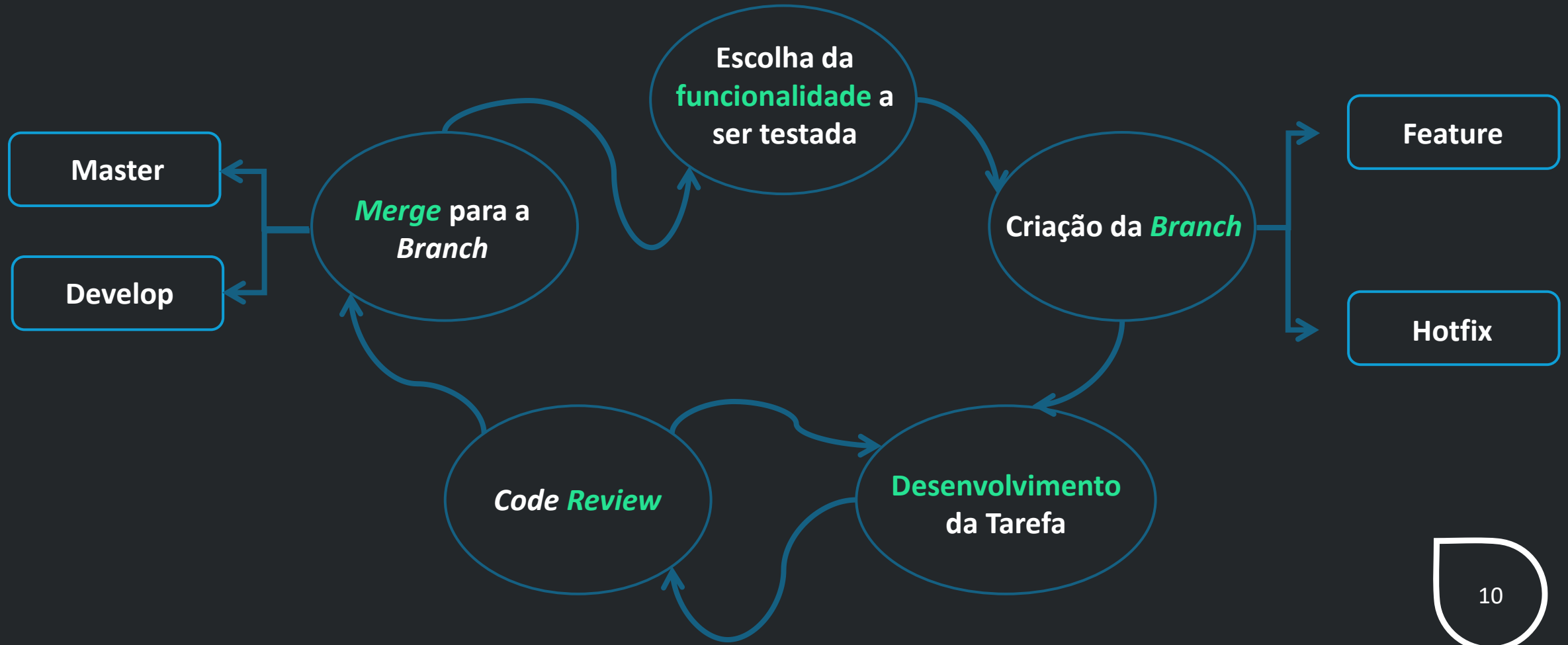
- Trabalho em equipa multidisciplinar (developers, QA, designers, PO);
- Comunicação próxima e contínua;

Rotina de Reuniões:

- **Reunião de Planeamento (início e final de sprint):**
  - Priorização de tarefas e definição de critérios de aceitação, retrospetiva das funcionalidades desenvolvidas em cada sprint
- **Daily Stand-up (diárias):**
  - Alinhar tarefas, identificar bloqueios, sincronizar esforços
- **All Hands (semanais):**
  - Alinhamento geral da empresa, partilha de novidades e objetivos

BRNDTS 2025

INTRODUÇÃO
Execução de Tarefas



BRNDTS 2025

DESENVOLVIMENTO
Formação

Formação teórica:

- Curso online **ISTQB Foundation Level V4.0 Complete Training;**
- Conceitos essenciais de QA, tipos de teste, ciclo de vida e boas práticas.

Estudo de ferramentas de teste:

- Cypress: estrutura de testes E2E, comandos, boas práticas;
- Cucumber e Gherkin: escrita de testes em linguagem natural (BDD).

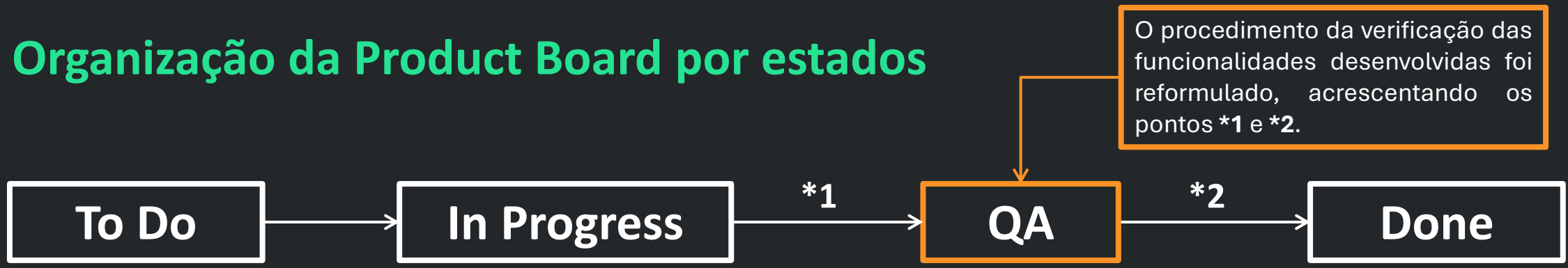
Gestão e rastreabilidade de testes:

- Zephyr Scale: criação, organização e acompanhamento de casos de teste;
- Integração com Jira para planeamento e análise de resultados.

DESENVOLVIMENTO

Adaptação da gestão de tarefas

Organização da Product Board por estados



***1** - Criação de um **card QA** exclusivo na Dev Board com workflow próprio:

- *In Progress → In Review → Done.*

- Validação manual e automatizada após a entrega da funcionalidade.

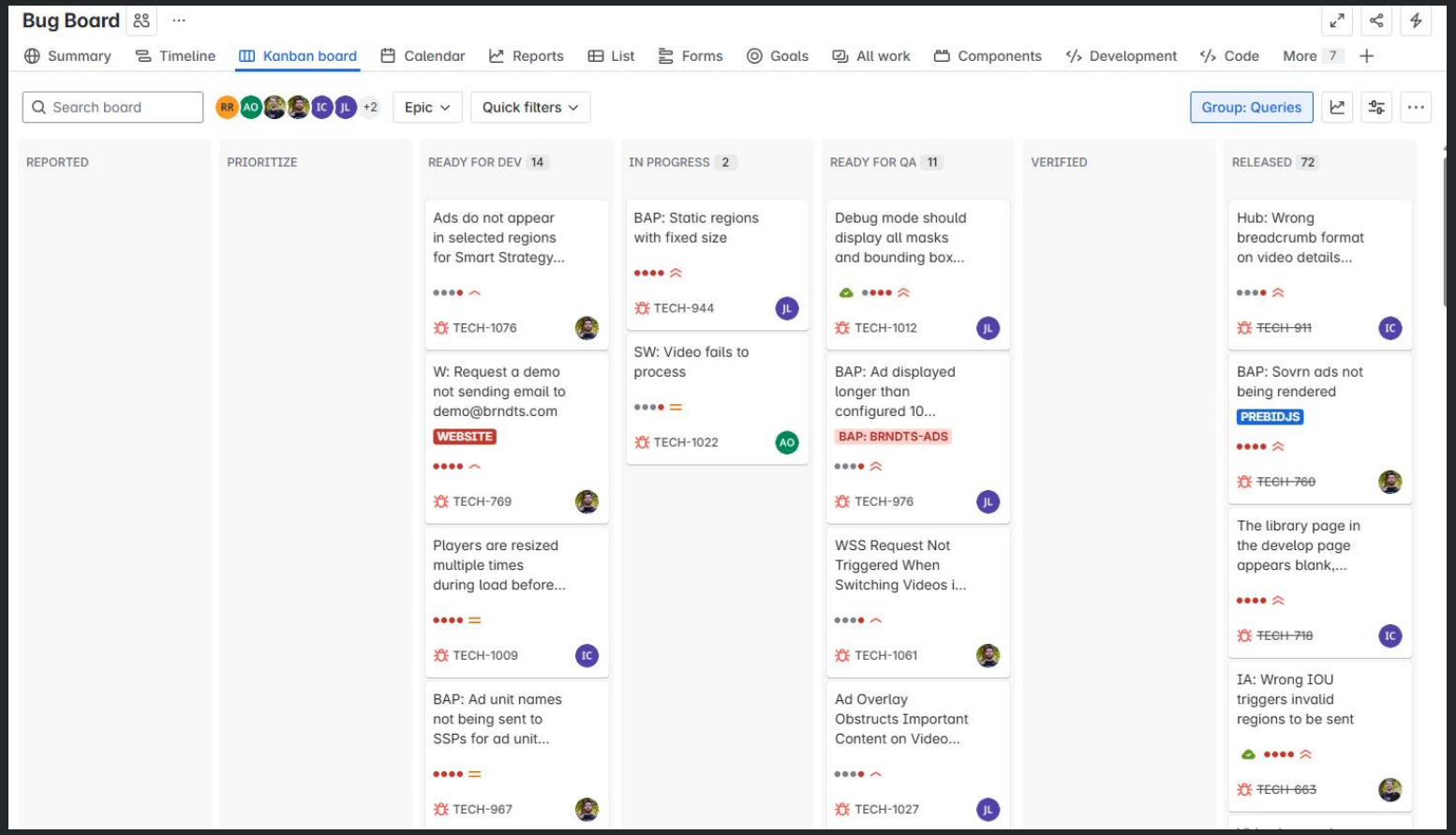
***2** - Após a validação da funcionalidade, o card da funcionalidade testada é movido para a coluna "Done".

BRNDTS 2025

DESENVOLVIMENTO

Identificação e gestão de bugs

Bug Board

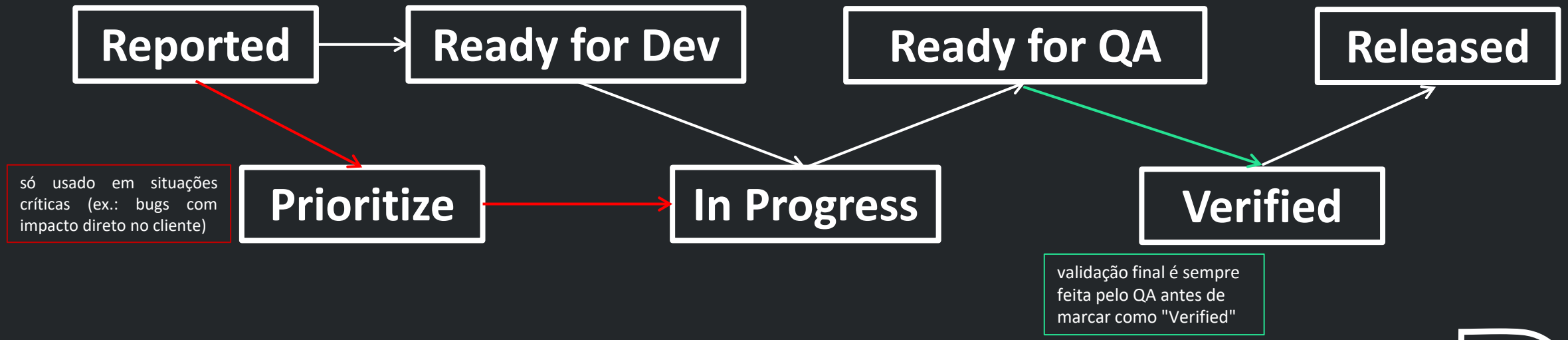


* Antes do estágio, a Bug Board existia, mas não tinha o devido valor nem era usada de forma estruturada, sendo maior parte dos bugs identificados registos esporádicos e sem detalhes.

DESENVOLVIMENTO

Identificação e gestão de bugs (cont.)

Fluxo de Gestão de Bugs (atualmente)



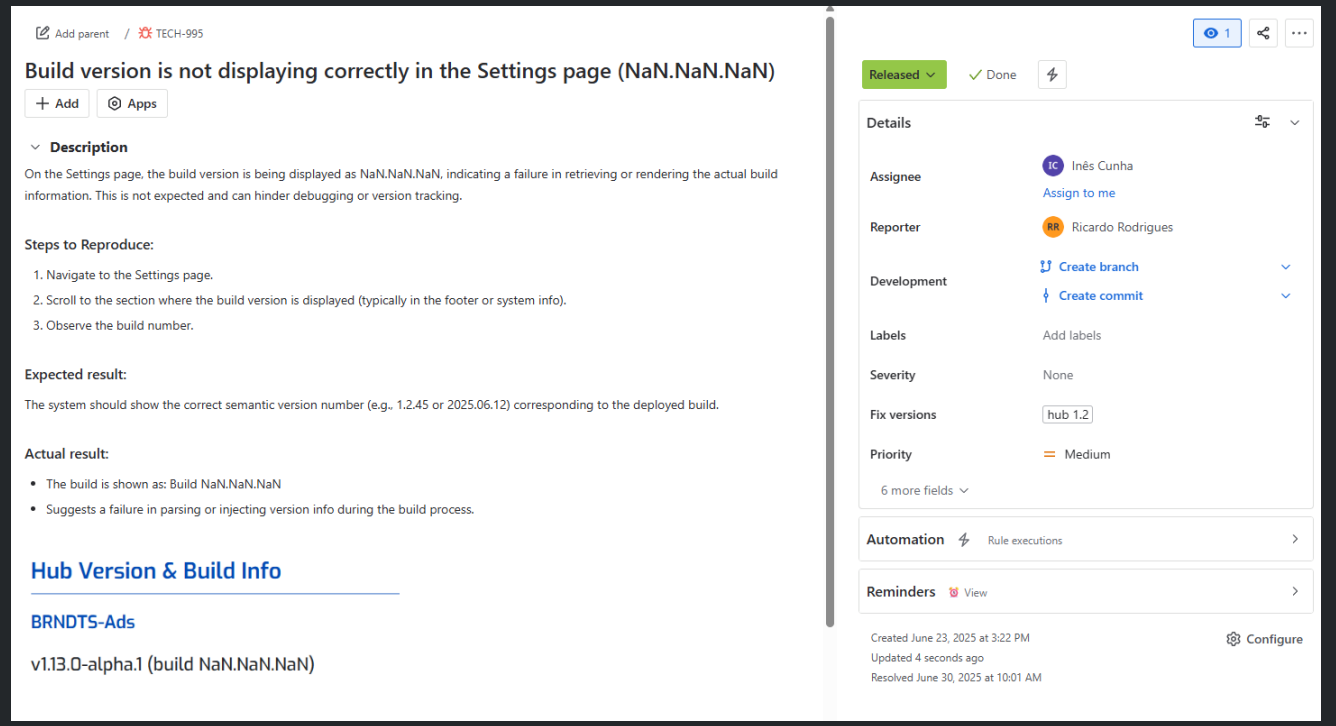
DESENVOLVIMENTO

Identificação e gestão de bugs (cont.)

Criação de Bug Cards

- Nome e descrição clara do problema;
- Passos para reproduzir o erro;
- Resultado esperado;
- Resultado obtido (atual);
- Evidência visual *(sempre que possível)*.

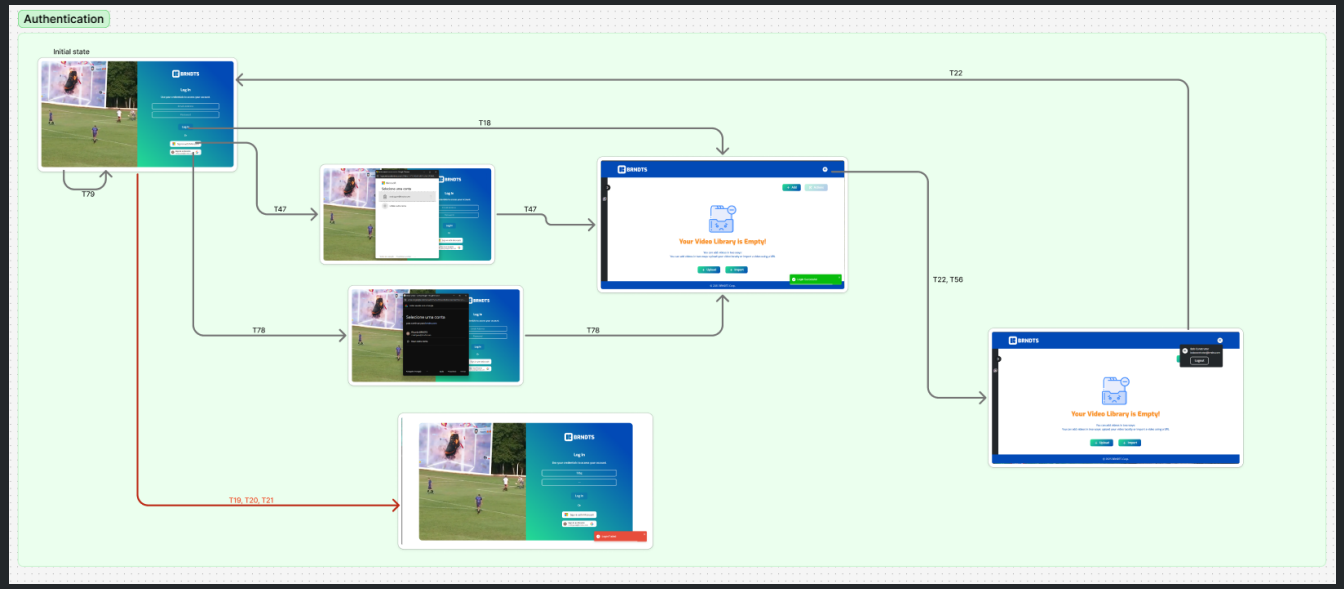
Todos os bugs são ligados a uma funcionalidade e release específica.



DESENVOLVIMENTO

Fluxo de testes e Automatização

- Os testes automatizados devem cobrir todo o percurso da funcionalidade a testar.
- Fluxogramas podem ajudar a visualizar o fluxo de iterações do utilizador e perceber as etapas a ser testadas.
- Cada transição no fluxograma deve ser representado por pelo menos um casos de teste (ex.: T18, T19...) descrito e implementado com BDD.



Fluxograma relativo aos casos de teste de Login e Logout

DESENVOLVIMENTO

Estrutura dos testes automatizados

Abordagem seguida:

- Frameworks: Cypress + Cucumber + TypeScript;
- Uso de BDD (Behavior Driven Development) + POM (Page Object Model)
- Separação clara entre:



- Colocando em prática nos casos de teste de Login:



DESENVOLVIMENTO

Estrutura dos testes automatizados (cont.)

Definição de Cenários:

- Utiliza a linguagem **Gherkin**, legível por humanos.
- Estrutura comum dos cenários:
  - **Given** – estado inicial (pré-condições);
  - **When** – ação realizada pelo utilizador;
  - **Then** – resultado esperado.
- Pode incluir:
  - **Background** – contexto comum a todos os cenários;
  - **Scenario Outline + Examples** – testes com múltiplos dados.

Login.feature

cypress > e2e > Hub > Authentication > Login.feature

```
Feature: Was login successful?
    As a user, I want to know if the login was successful

    Background:
        Given I open the login page

    Scenario Outline: [<T#>] Login with credentials was or was not successful

        When I enter username or email "<email>"
            And  I enter password "<password>"
            And I click the login button
        Then I should be told "<result>"

    @smoke
    Examples:
        | T#  | email    | password  | result  |
        | T18 | bob      | zxcqweasd | success |
        | T20 | anything | anything  | failure |
        | T19 | bob      | anything  | failure |
        | T21 |          |           | failure |

    Scenario: [T79] All authentication options (Custom credentials, Microsoft, Google) are or are not visible
        Then The custom credentials option should be visible
        Then The Microsoft option should be visible
        Then The Google option should be visible

```

Implementação de BDD no Login.feature

DESENVOLVIMENTO

Estrutura dos testes automatizados (cont.)

Implementação dos passos:

- Cada linha do .feature tem associada funções reais que interagem com a aplicação.
- Utiliza os métodos definidos no Page Actions.

Login.ts

cypress > e2e > Hub > Authentication > Login.ts > ...

```ts
import { When, Then, Given } from "@badeball/cypress-cucumber-preprocessor";
import { LoginPage } from "../../../support/pages";

const loginPage = new LoginPage();

Given("I open the login page", function () {
  loginPage.go();
});

When("I enter username or email {string}", function (email: string) {
  email && loginPage.fillUsername(email);
});

When("I enter password {string}", function (password: string) {
  password && loginPage.fillPassword(password);
});

When("I click the login button", function () {
  loginPage.clickLoginButton();
});

Then("I should be told {string}", function (expected: "success" | "failure") {
  cy.wait(500);
  loginPage.checkNotification(
    expected === "success" ? "success" : "error",
    expected === "success" ? "Login Successful" : "Login Failed"
  );
  loginPage.closeNotification();
});

Then("The custom credentials option should be visible", function () {
  loginPage.checkCustomCredentialsOption();
});

Then("The Microsoft option should be visible", function () {
  loginPage.checkMicrosoftOption();
});

Then("The Google option should be visible", function () {
  loginPage.checkGoogleOption();
});

```

Implementação dos passos no Login.ts

DESENVOLVIMENTO

Estrutura dos testes automatizados (cont.)

Funções de interação (Page Actions):

- Contém métodos reutilizáveis que encapsulam ações sobre a UI;
- Facilita a manutenção dos testes e reduz repetição de código;
- Exemplo: preencher o campo de email, clicar no botão de login.

LoginPage.ts

cypress > support > pages > Login > LoginPage.ts > ...

```ts
import { BRNDTSPage } from "../BRNDTSPage";
import { LoginElements } from "./LoginElements";

export class LoginPage extends BRNDTSPage {
  go() {
    cy.visit("/login", { failOnStatusCode: false });
    cy.wait(1000);
    return this;
  }

  fillUsername(value: string) {
    cy.get(LoginElements.usernameInput).clear().type(value);
    return this;
  }

  fillPassword(value: string) {
    cy.get(LoginElements.passwordInput).clear().type(value);
    return this;
  }

  clickLoginButton() {
    cy.get(LoginElements.loginButton).click();
    return this;
  }

  checkCustomCredentialsOption() {
    cy.get(LoginElements.usernameInput).should("be.visible");
    cy.get(LoginElements.passwordInput).should("be.visible");
    cy.get(LoginElements.loginButton).should("be.visible");
    return this;
  }

  checkMicrosoftOption() {
    cy.get(LoginElements.microsoftButton).should("be.visible");
    return this;
  }

  checkGoogleOption() {
    cy.get(LoginElements.googleButton).should("be.visible");
    return this;
  }
}
```

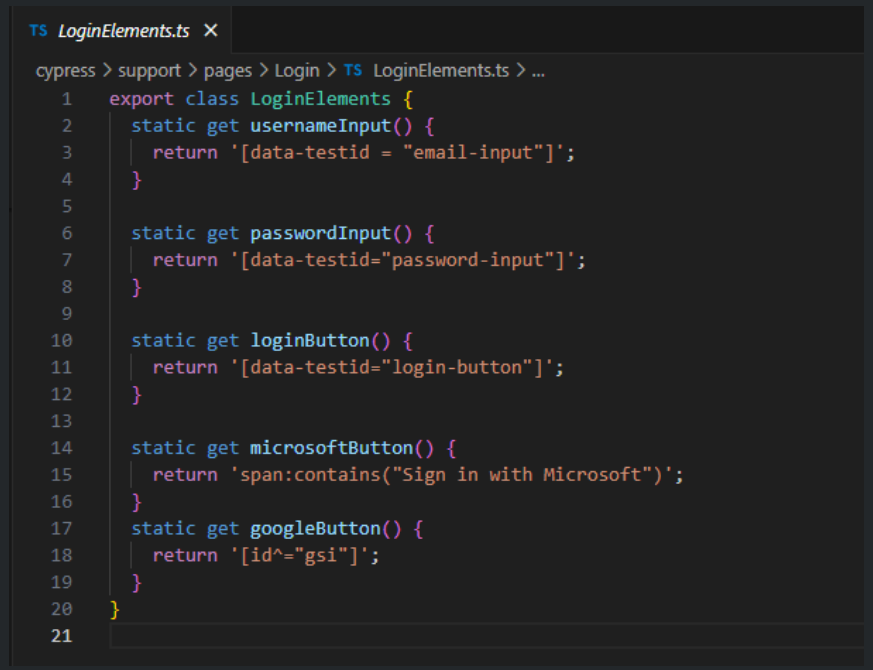
Funções de interação no LoginPage.ts

DESENVOLVIMENTO

Estrutura dos testes automatizados (cont.)

Elementos da Interface (Page Objects):

- Centraliza os elementos HTML utilizados nos testes (ex.: data-testid);
- Evita duplicação e facilita alterações.



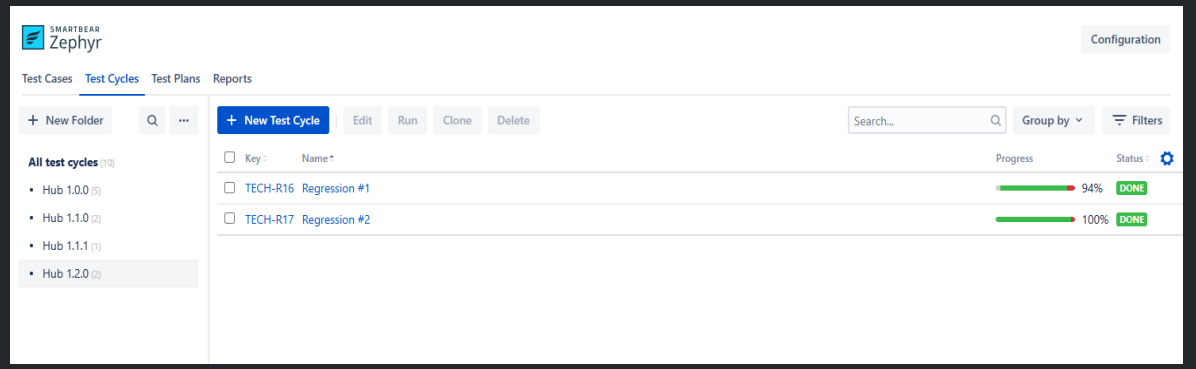
Elementos da interface no LoginElements.ts

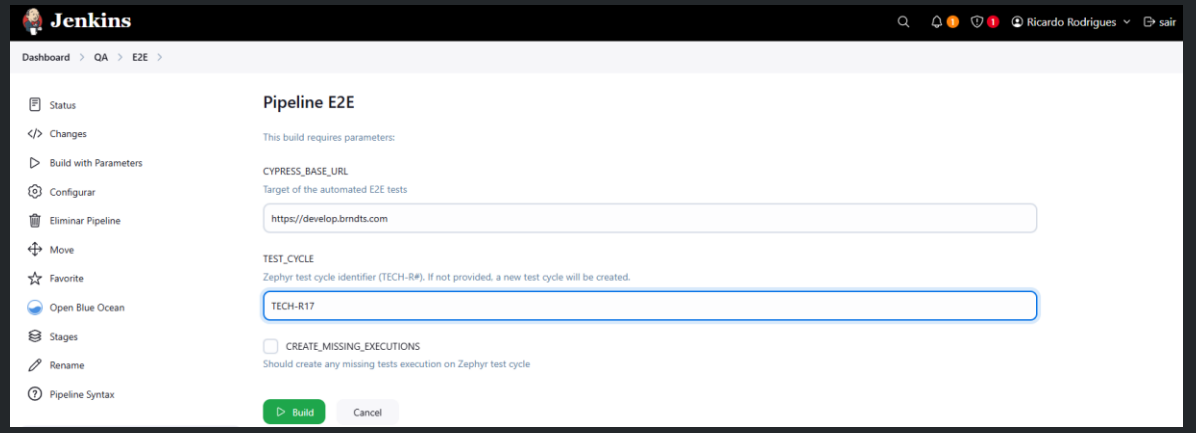
DESENVOLVIMENTO

Execução das pipelines CI/CD

Configuração dos test cycles e pipelines:

- Um Test Cycle é previamente criado no Zephyr Scale, com os testes relevantes para a release;
- Esse ciclo gera um ID único, que é utilizado na configuração da pipeline no Jenkins.





DESENVOLVIMENTO

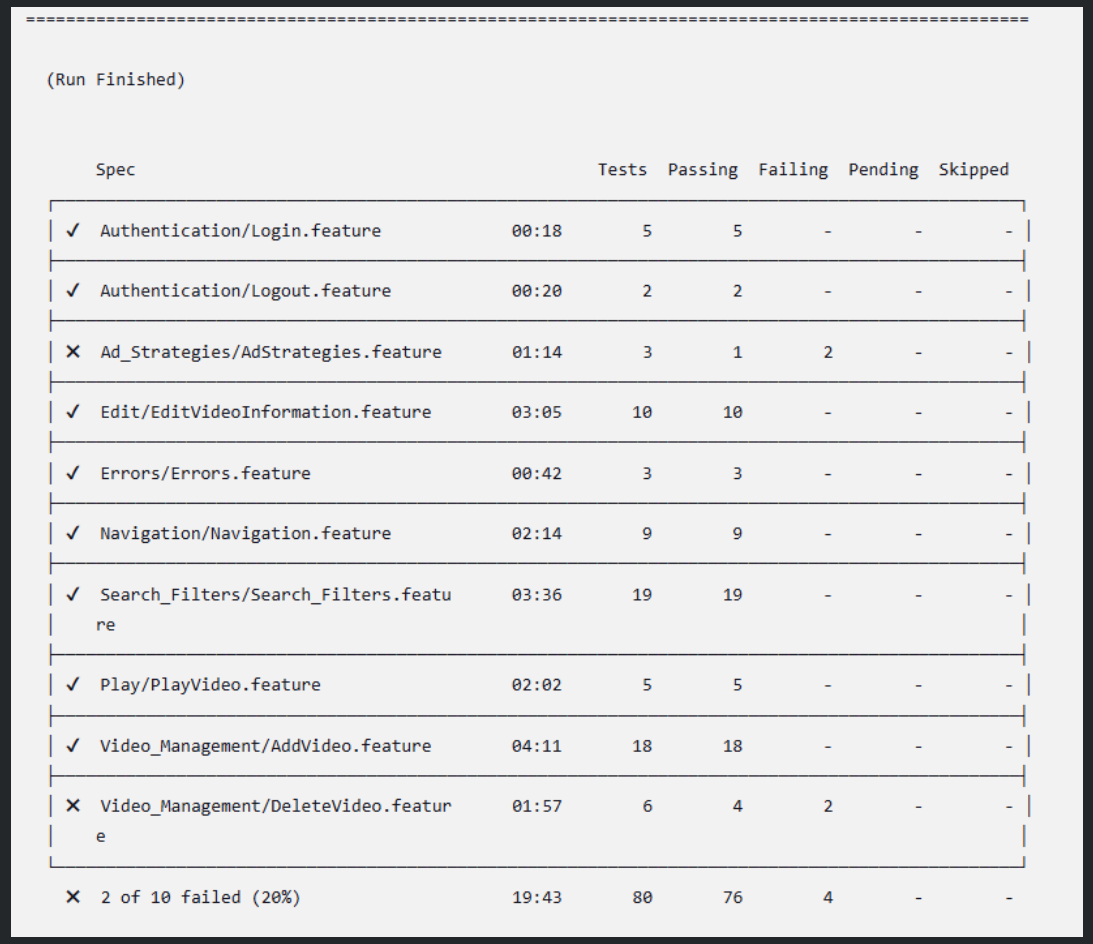
Execução das pipelines CI/CD (cont.)

Execução da pipeline:

- Jenkins executa todos os testes E2E definidos no repositório de QA da empresa, em modo headless do Cypress;
- No final da execução, apresenta pequenos relatórios sobre os resultados dos testes.

Melhoria futura:

- Execução apenas dos testes presentes no test cycle configurado no Zephyr.



(Run Finished)

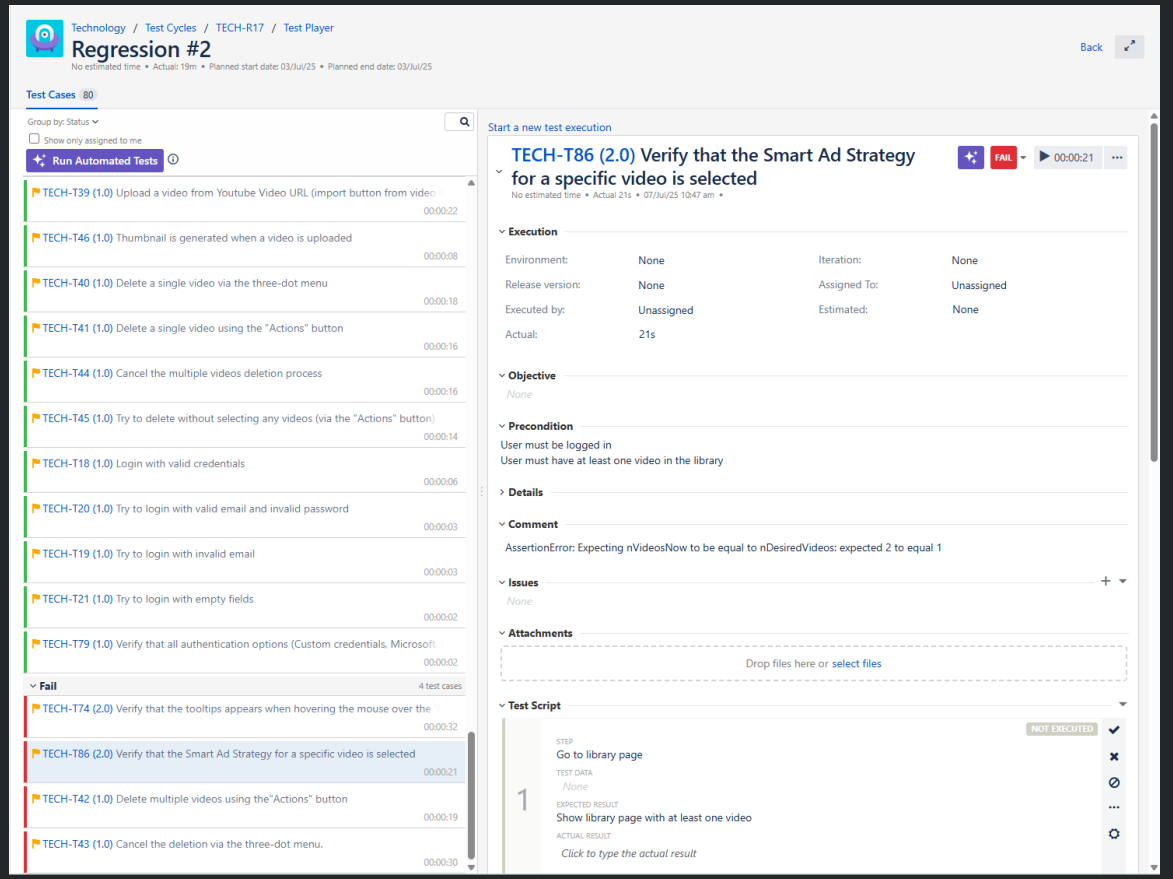
	Spec		Tests	Passing	Failing	Pending	Skipped
✓	Authentication/Login.feature	00:18	5	5	-	-	-
✓	Authentication/Logout.feature	00:20	2	2	-	-	-
✗	Ad_Strategies/AdStrategies.feature	01:14	3	1	2	-	-
✓	Edit/EditVideoInformation.feature	03:05	10	10	-	-	-
✓	Errors/Errors.feature	00:42	3	3	-	-	-
✓	Navigation/Navigation.feature	02:14	9	9	-	-	-
✓	Search_Filters/Search_Filters.feature	03:36	19	19	-	-	-
✓	Play/PlayVideo.feature	02:02	5	5	-	-	-
✓	Video_Management/AddVideo.feature	04:11	18	18	-	-	-
✗	Video_Management/DeleteVideo.feature	01:57	6	4	2	-	-
✗	2 of 10 failed (20%)	19:43	80	76	4	-	-

DESENVOLVIMENTO

Execução das pipelines CI/CD (cont.)

Resultados obtidos:

- Após a execução, os resultados dos testes são automaticamente enviados para o Zephyr Scale;
- Os resultados são associados diretamente ao Test Cycle criado, permitindo análise centralizada.

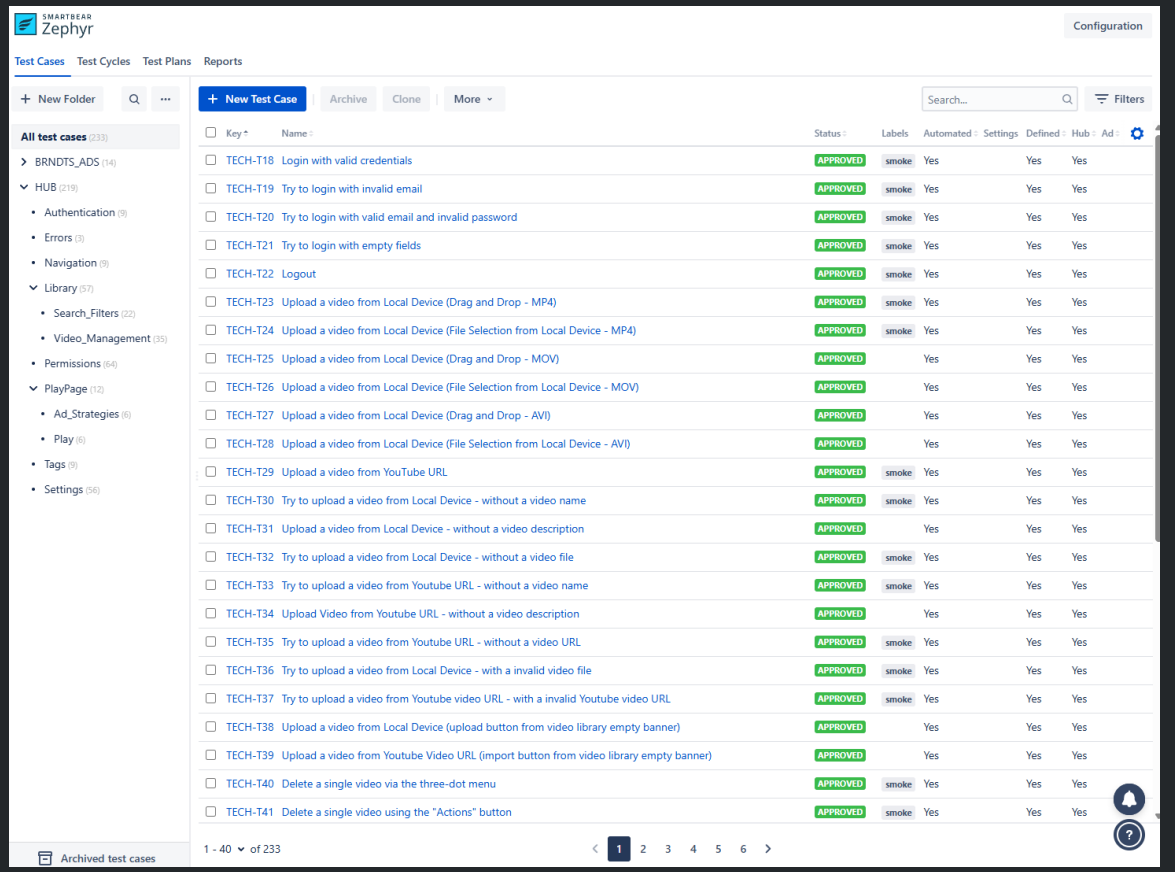


CONCLUSÃO

Resultados da estratégia desenvolvida

Criação de casos de teste

- Ao longo do estágio, foram criados um total de 233 casos de teste, distribuídos entre casos automatizados e casos ainda testados manualmente.
- Campos customizáveis foram usados como forma de garantir uma melhor organização.



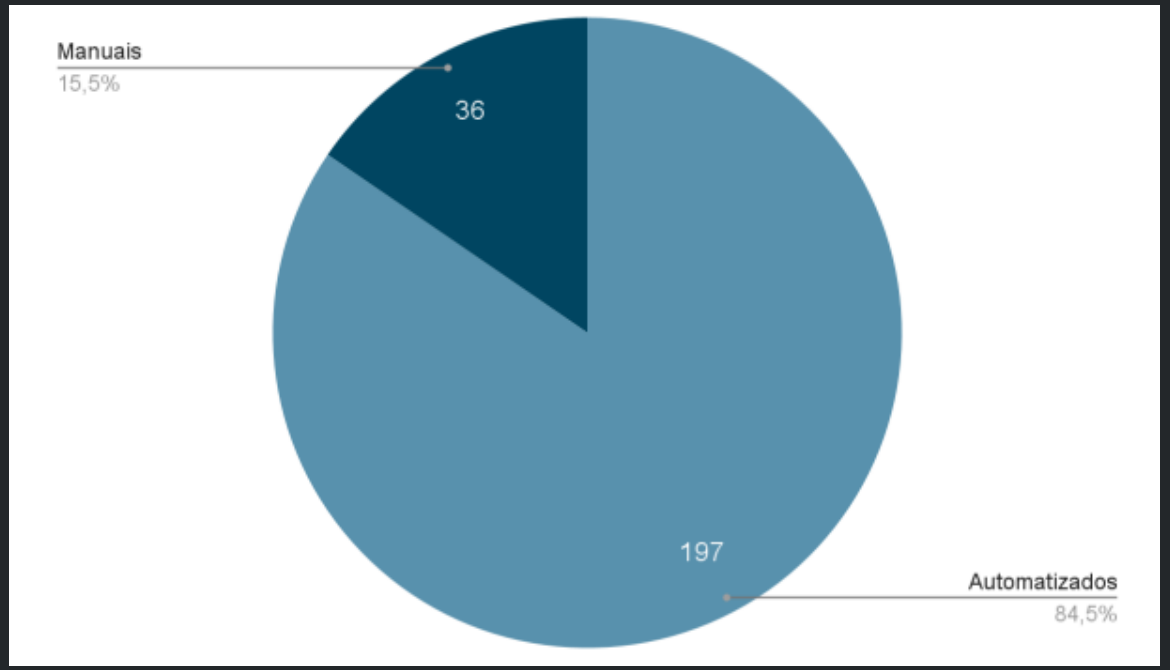
CONCLUSÃO

Resultados da estratégia desenvolvida (cont.)

Taxa de automatização

Dos 233 casos de teste criados, a taxa de automatização é a seguinte:

- 197 casos automatizados (84,5%);
- 36 ainda por automatizar, sendo executados manualmente (15,5%).



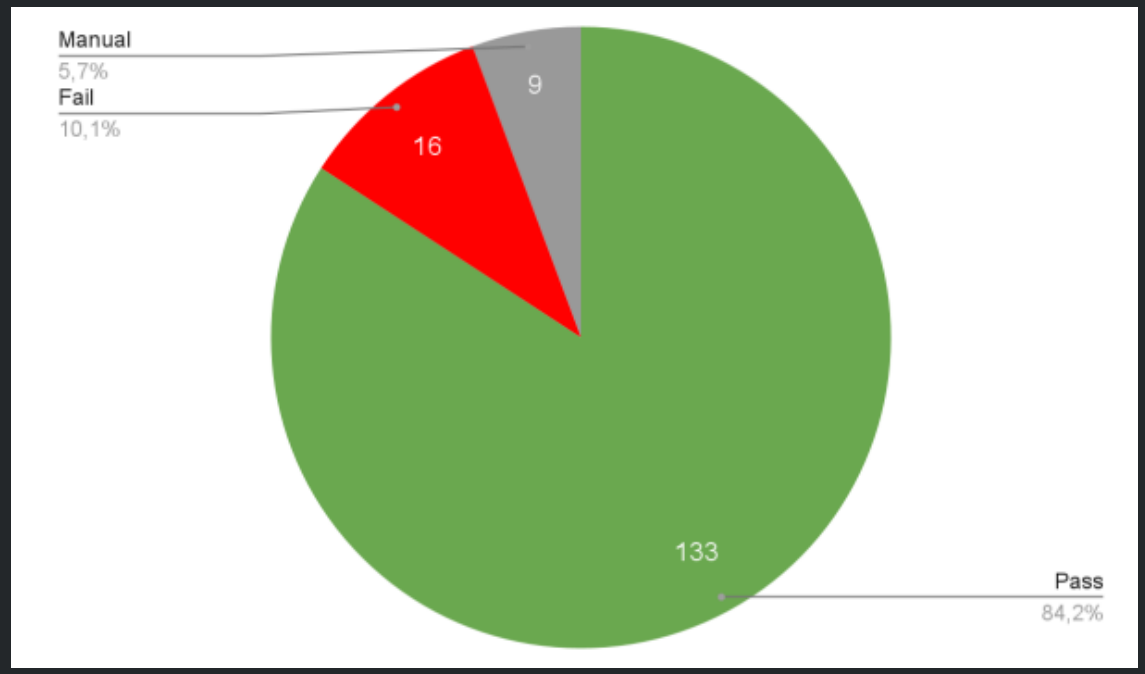
CONCLUSÃO

Resultados da estratégia desenvolvida (cont.)

Resultados de Test Cycles

Numa das últimas execuções de um test cycle, foram testados 158 casos de teste com os seguintes resultados:

- 133 passaram (84.2%);
- 16 falharam (10.1%);
- 9 não estavam automatizados (5.7%), tendo sido executados manualmente.



BRNDTS 2025

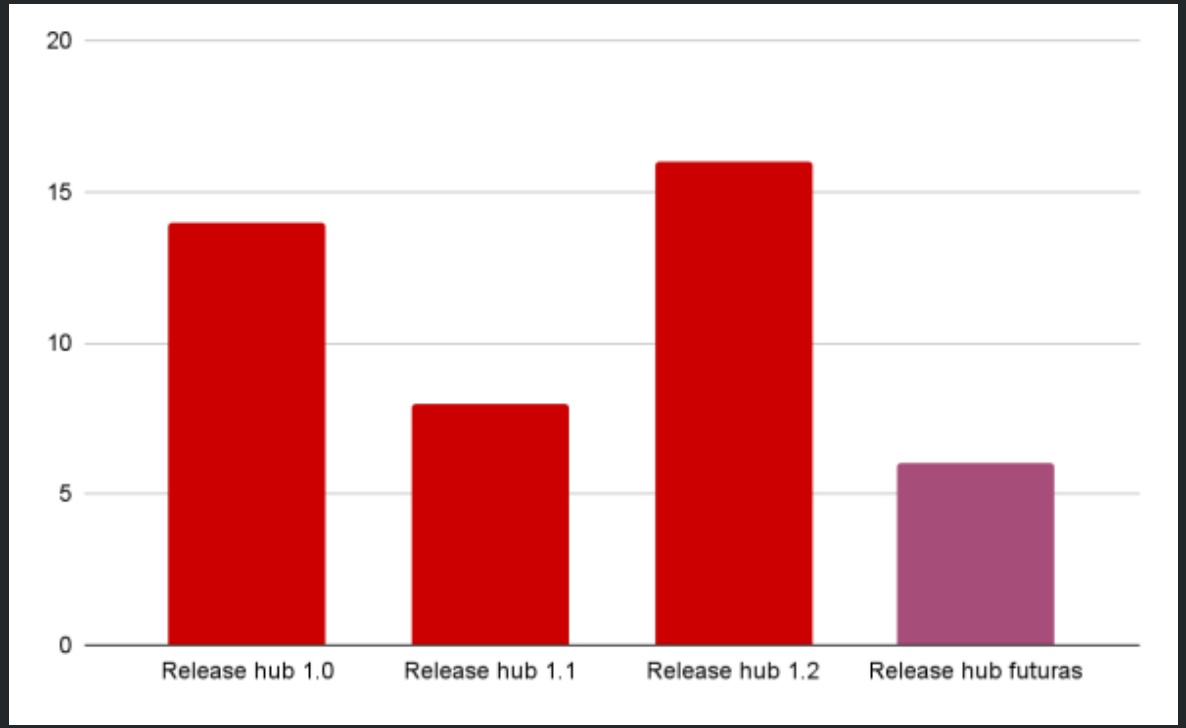
CONCLUSÃO

Resultados da estratégia desenvolvida (cont.)

Bugs identificados

Durante o estágio, foram identificados **44 bugs** em **3 releases** distintas, todos criados como bug cards para correção pela equipa de desenvolvimentos, sendo distribuídos por:

- 14 bugs identificados e corrigidos na Release Hub 1.0;
- 8 bugs identificados e associados na Release Hub 1.1;
- 16 bugs identificados e associados na Release Hub 1.2;
- 6 bugs identificados na Release Hub 1.2, mas que serão resolvidos em Releases Hub futuras.



CONCLUSÃO

Reflexão critica da estratégia implementada

O que correu bem:

- Cobertura de testes automatizados de 84,5%;
- Redução significativa do esforço manual em testes de regressão;
- Aumento da confiança da equipa de desenvolvimento nas releases;
- Deteção precoce de bugs, evitando problemas em produção;
- Melhoria real na estabilidade e qualidade do produto.

Limitações:

- Configurações das pipelines CI/CD com Jenkins não concluída (por dependências técnicas);
- Alguns testes ainda não automatizados.

Sugestões futuras:

- Consolidar o papel do QA no fluxo de desenvolvimento;
- Expandir testes automatizados e segmentar execuções;
- Criar documentação de boas práticas para continuidade.

BRN DTS

Obrigado pela atenção